

Worksheet No. – 6

Name: Harsh

UID: 24MCA20045

Branch: MCA

Section/Group: 24MCA-1B

Semester: 3rd

Date of Performance: 28-10-25

Subject Name: Backend Technologies LAB

Subject Code: 24CAP-709

Aim/Overview of Practical:

Authenticate the student who is checking marks from university portal as created in experiment.

Objectives:

- 1) To authenticate students using **JWT tokens**.
- 2) To protect passwords using **bcrypt hashing**.
- 3) To store and manage student data in **MongoDB**.

Project Structure

university-portal-node/

```
|— public/
|   |— index.html
|   |— login.html
|   |— student.html
|   |— teacher.html
|— models/
|   |— Attendance.js
|   |— Marks.js
|   |— Student.js
|   |— user.js
```

```
└── routes/  
    | └── api.js  
└── server.js
```

Input/Code:

App.js

```
// routes/api.js  
const express = require('express');  
const router = express.Router();  
const bcrypt = require('bcrypt');  
const User = require('../models/User');  
const Student = require('../models/Student');  
const Mark = require('../models/Mark');  
const Attendance = require('../models/Attendance');  
function requireAuth(req, res, next){  
  if (!req.session.user) return res.status(401).json({ error: 'Not authenticated' });  
  next();  
}  
function requireTeacher(req, res, next){  
  if (!req.session.user || req.session.user.role !== 'teacher') return res.status(403).json({ error: 'Forbidden' });  
  next();  
}  
function requireStudent(req, res, next){  
  if (!req.session.user || req.session.user.role !== 'student') return res.status(403).json({ error: 'Forbidden' });  
  next();  
}  
router.post('/signup', async (req, res) => {  
  try {  
    const { username, password, full_name, role, roll_no } = req.body;  
  
    if (!username || !password || !role) {  
      return res.status(400).json({ error: 'Missing required fields' });  
    }  
    if (!['student', 'teacher'].includes(role)) {  
      return res.status(400).json({ error: 'Invalid role' });  
    }  
    const exists = await User.findOne({ username });  
    if (exists) return res.status(409).json({ error: 'Username already exists' });  
    const hashed = await bcrypt.hash(password, 10);  
    let student_id = null;  
    if (role === 'student') {  
      const lastStudent = await Student.findOne({}).sort({ student_id: -1 }).lean();  
      student_id = lastStudent ? (lastStudent.student_id + 1) : 1;  
    }  
    const user = await User.create({  
      username,  
      password: hashed,
```

```

    role,
    full_name: full_name || username,
    student_id: student_id
  });
  if (role === 'student') {
    await Student.create({
      student_id,
      roll_no: roll_no || `R${String(student_id).padStart(3, '0')}`,
      user: user._id
    });
  }
  req.session.user = {
    id: user._id.toString(),
    username: user.username,
    role: user.role,
    full_name: user.full_name,
    student_id: user.student_id || null
  };
  return res.json({ success: true, role: user.role });
} catch (err) {
  console.error('Signup error', err);
  return res.status(500).json({ error: 'Server error' });
}
});
router.post('/login', async (req, res) => {
  try {
    const { username, password } = req.body;
    if (!username || !password) return res.status(400).json({ error: 'Missing credentials' });
    const user = await User.findOne({ username });
    if (!user) return res.status(401).json({ error: 'Invalid credentials' });
    const match = await bcrypt.compare(password, user.password);
    if (!match) return res.status(401).json({ error: 'Invalid credentials' });
    req.session.user = {
      id: user._id.toString(),
      username: user.username,
      role: user.role,
      full_name: user.full_name,
      student_id: user.student_id || null
    };
    res.json({ success: true, role: user.role });
  } catch (err) {
    console.error('Login error', err);
    res.status(500).json({ error: 'Server error' });
  }
});
router.post('/logout', (req, res) => {
  req.session.destroy(err => {
    if (err) {
      console.error('Logout error', err);
      return res.status(500).json({ error: 'Could not logout' });
    }
  });
});

```

```
res.clearCookie && res.clearCookie('connect.sid'); // in case
return res.json({ success: true });
});
});
router.get('/me', (req, res) => {
  res.json({ user: req.session.user || null });
});
router.get('/marks', requireStudent, async (req, res) => {
  try {
    const studentId = req.session.user.student_id;
    const marks = await Mark.find({ student_id: studentId }).lean();
    res.json({ marks });
  } catch (err) {
    console.error('Marks fetch error', err);
    res.status(500).json({ error: 'Server error' });
  }
});
router.get('/attendance', requireAuth, async (req, res) => {
  try {
    if (req.session.user.role === 'student') {
      const studentId = req.session.user.student_id;
      const attendance = await Attendance.find({ student_id: studentId }).sort({ date: -1 }).lean();
      return res.json({ attendance });
    } else {
      if (req.query.student_id) {
        const studentId = Number(req.query.student_id);
        const attendance = await Attendance.find({ student_id }).sort({ date: -1 }).lean();
        return res.json({ attendance });
      } else {
        const attendance = await Attendance.find({}).sort({ date: -1 }).limit(100).lean();
        return res.json({ attendance });
      }
    }
  } catch (err) {
    console.error('Attendance fetch error', err);
    res.status(500).json({ error: 'Server error' });
  }
});
router.post('/marks/add', requireTeacher, async (req, res) => {
  try {
    const { student_id, subject, marks, semester } = req.body;
    if (!student_id || !subject || marks === null) {
      return res.status(400).json({ error: 'Missing fields' });
    }
    const newMark = await Mark.create({
      student_id: Number(student_id),
      subject: String(subject),
      marks: Number(marks),
      semester: semester ? Number(semester) : 1
    });
  }
});
```

```

    res.json({ success: true, mark: newMark });
  } catch (err) {
    console.error('Add mark error', err);
    res.status(500).json({ error: 'Server error' });
  }
});
router.get('/marks/student/:student_id', requireTeacher, async (req, res) => {
  try {
    const studentId = Number(req.params.student_id);
    const marks = await Mark.find({ student_id: studentId }).lean();
    res.json({ marks });
  } catch (err) {
    console.error('Get marks for student error', err);
    res.status(500).json({ error: 'Server error' });
  }
});
module.exports = router;

```

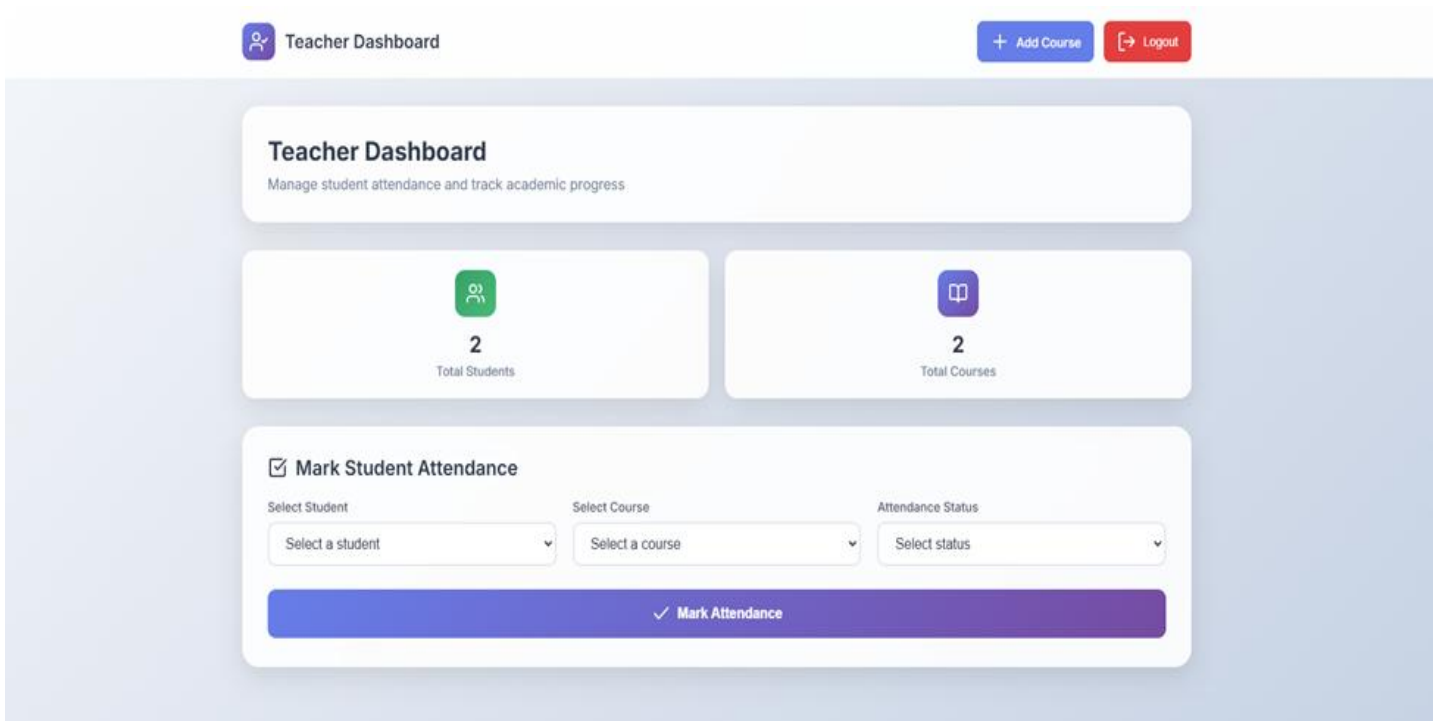
attendance.js

```

const mongoose = require('mongoose');
const Schema = mongoose.Schema;
const attendanceSchema = new Schema({
  student_id: { type: Number, required: true },
  date: { type: Date, required: true },
  status: { type: String, enum: ['present', 'absent'], required: true },
  marked_by: { type: Schema.Types.ObjectId, ref: 'User' }
});
module.exports = mongoose.model('Attendance', attendanceSchema);

```

Output/Result:



The screenshot shows a 'Teacher Dashboard' interface. At the top, there's a header with a user icon, the text 'Teacher Dashboard', and two buttons: '+ Add Course' and 'Logout'. Below the header, the main content area has a title 'Teacher Dashboard' with the subtitle 'Manage student attendance and track academic progress'. There are two summary cards: 'Total Students' with a value of 2 and 'Total Courses' with a value of 2. Below these is a section titled 'Mark Student Attendance' which includes three dropdown menus: 'Select Student' (with 'Select a student' as the placeholder), 'Select Course' (with 'Select a course' as the placeholder), and 'Attendance Status' (with 'Select status' as the placeholder). At the bottom of this section is a large blue button labeled 'Mark Attendance' with a checkmark icon.



Welcome Back

Sign in to your attendance account

Email Address

harshjangid33815@gmail.com

Password

Login as

Student

Sign In

Don't have an account? [Create one here](#)



Welcome Back

Sign in to your attendance account

Email Address

harshjangid33815@gmail.com

Password

Login as

Teacher

Sign In

Don't have an account? [Create one here](#)



Student Dashboard

Logout

My Attendance

Track your attendance records and academic progress



2

Total Days



1

Present Days



1

Absent Days

Attendance Records

COURSE	DATE	STATUS
Backend technologies	Nov 1, 2025	PRESENT
frontend	Nov 1, 2025	ABSENT

exp 3 > Ex6 > students

Documents 2 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [EXPORT DATA](#) [UPDATE](#) [DELETE](#)

```
{
  "_id": ObjectId('6904a205512fed21a1621626'),
  "name": "Varun",
  "email": "varun@gmail.com",
  "password": "$2b$10$IsDcmmP00J6WF9KawPri/ea0Bk208Z/T89Top3DI5h46jbMhxVYpy",
  "__v": 0
}
```

```
{
  "_id": ObjectId('6904ee9c25f87f3977254ae7'),
  "name": "harsh",
  "email": "harshjangid33815@gmail.com",
  "password": "$2b$10$YHt61wL56kYvsJ8xepPGfew0D4uRv3GCOvJmDtS4gcQ/1BA92XtNW",
  "__v": 0
}
```

exp 3 > Ex6 > attendances

Documents 5 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [EXPORT DATA](#) [UPDATE](#) [DELETE](#)

```
{
  "_id": ObjectId('6904a22e512fed21a1621631'),
  "student": ObjectId('6904a205512fed21a1621626'),
  "course": ObjectId('6904a229512fed21a162162d'),
  "status": "Present",
  "markedBy": ObjectId('6904a1f8512fed21a1621623'),
  "date": 2025-10-31T11:49:02.835+00:00,
  "__v": 0
}
```

```
{
  "_id": ObjectId('6904f36525f87f3977254b02'),
  "student": ObjectId('6904a205512fed21a1621626'),
  "course": ObjectId('6904a229512fed21a162162d'),
  "status": "Present",
  "markedBy": ObjectId('6904a1f8512fed21a1621623'),
  "date": 2025-10-31T17:35:33.736+00:00,
  "__v": 0
}
```

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [EXPORT DATA](#) [UPDATE](#) [DELETE](#)

```
{
  "_id": ObjectId('6904a229512fed21a162162d'),
  "name": "Backend technologies",
  "__v": 0
}
```

```
{
  "_id": ObjectId('6904eebe25f87f3977254af0'),
  "name": "frontend",
  "__v": 0
}
```

exp 3 > Ex6 > teachers

Documents 1 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

[ADD DATA](#) [EXPORT DATA](#) [UPDATE](#) [DELETE](#)

```
{
  "_id": ObjectId('6904a1f8512fed21a1621623'),
  "name": "Harsh",
  "email": "harshjangid33815@gmail.com",
  "password": "$2b$10$jf47B9P9HTW60ZVQ0wj0dugtXac9dgs/lXRkS0No16kwmZR0Y7obw",
  "__v": 0
}
```

Learning outcomes:

1. Learned to implement **student authentication** using **JWT**.
2. Understood **password hashing** with **bcrypt.js** for security.
3. Gained knowledge of using **middleware** for token verification.
4. Learned to create **protected routes** accessible only to authenticated users.